Université Abou Bekr Belkaid – Tlemcen
Faculté des Sciences
Déartement d'Informtique

Année Universitaire : 2025-2026

Module : Traitement d'Image &Vision par Ordinateur

# Rapport de TP 1

**Douache Ahmed Yacine**

**16/ 03 / 2026**

## Exercice 1 Partie 1:

### 1. Problème

Dans cet exercice, l'objectif est d'analyser et d'améliorer le **contraste des images**.
Pour cela, nous devons charger trois images (`aqui.pgm`, `pont.pgm`, `Loup-noir.jpg`), construire leur **histogramme**, trouver les valeurs minimale et maximale des pixels, puis appliquer un **étirement d'histogramme** pour améliorer la qualité de l'image.

Ensuite, nous évaluons le contraste de l'image en utilisant deux méthodes : **Michelson** et **RMS**.

### 2. Solution

La solution consiste à suivre les étapes suivantes :

1. Charger les images en **niveau de gris** avec la bibliothèque OpenCV.
2. Afficher l'histogramme des images avec Matplotlib afin d'observer la distribution des niveaux de gris.
3. Calculer les valeurs **minimale** et **maximale** des pixels.
4. Appliquer un **étirement d'histogramme** pour utiliser toute la plage des niveaux de gris.
5. Sauvegarder les nouvelles images obtenues.
6. Calculer les mesures de contraste **Michelson** et **RMS** pour analyser l'amélioration du contraste.

### 3. Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# list des chemins d'images à traiter
images = [
    "D:/TP VO/TP1/aqui.pgm",
    "D:/TP VO/TP1/pont.pgm",
    "D:/TP VO/TP1/Loup-noir.jpg"
]
```

```python
# boucle pour traiter chaque image
for path in images:

    print("\nTraitement de :", path)
    img = cv2.imread(path, 0)

    if img is None:
        print("Erreur : image non trouvée")
        continue

    # affichage de l'histogramme original
    plt.figure()
    plt.hist(img.ravel(), bins=256)
    plt.title("Histogramme original")
    plt.show()

    # calcul de Imin et Imax
    Imin = img.min()
    Imax = img.max()

    print("Imin =", Imin)
    print("Imax =", Imax)

    # étirement de l'histogramme
    img_stretch = (img - Imin) * (255 / (Imax - Imin))
    img_stretch = img_stretch.astype(np.uint8)

    # sauvegarde de l'image étirée
    name = path.split("/")[-1]
    output_name = "etire_" + name
    cv2.imwrite(output_name, img_stretch)

    # affichage de l'histogramme étiré
    plt.figure()
    plt.hist(img_stretch.ravel(), bins=256)
    plt.title("Histogramme étiré")
    plt.show()

   # calcul du contraste de Michelson
    michelson = (Imax - Imin) / (Imax + Imin)
    print("Contraste Michelson =", michelson)


    mean = np.mean(img)

    rms = np.sqrt(np.mean((img - mean)**2))
```

```
    print("Contraste RMS =", rms)
```

**4. Résultats**

Les résultats montrent que :

- l'histogramme avant traitement est souvent concentré sur une petite plage de niveaux de gris
- après l'étirement, l'histogramme est mieux réparti
- les images obtenues sont plus claires et présentent un meilleur contraste

Les nouvelles images sont sauvegardées après le traitement.

**5. Commentaires et conclusion**

L'étirement d'histogramme permet d'améliorer le contraste global d'une image de manière simple et efficace.
Cette méthode est particulièrement utile lorsque les niveaux de gris d'une image sont concentrés dans une petite plage.

Cependant, pour certaines images, des méthodes plus avancées comme l'égalisation d'histogramme peuvent donner de meilleurs résultats.

## Exercice 2 Partie 1:

1. **Problème**

Dans cet exercice, l'objectif est d'améliorer le **contraste des images** en utilisant la technique de **l'égalisation d'histogramme**.

Nous devons charger les images `aqui.pgm`, `pont.pgm` et `Loup-noir.jpg`, construire leur histogramme puis appliquer une égalisation afin d'obtenir une meilleure distribution des niveaux de gris.

Ensuite, nous évaluons le résultat en comparant les images avant et après traitement et en calculant les mesures de contraste **Michelson** et **RMS**

2. **Solution**

La solution consiste à suivre les étapes suivantes :

1. Charger les images en niveau de gris avec la bibliothèque OpenCV.
2. Afficher l'histogramme de chaque image avec Matplotlib.
3. Calculer l'histogramme cumulé de l'image.
4. Appliquer l'égalisation d'histogramme pour redistribuer les niveaux de gris.
5. Sauvegarder la nouvelle image obtenue.
6. Calculer les mesures de contraste Michelson et RMS pour évaluer l'amélioration.

### 3. Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Liste des images
images = [
    "D:/TP VO/TP1/aqui.pgm",
    "D:/TP VO/TP1/pont.pgm",
    "D:/TP VO/TP1/Loup-noir.jpg"
]

for path in images:

    print("Traitement de :", path)

    # Charger image en gris
    img = cv2.imread(path, 0)

    if img is None:
        print("Erreur chargement")
        continue

    # Histogramme original
    plt.hist(img.ravel(), 256)
    plt.title("Histogramme original")
    plt.show()

    # Egalisation d'histogramme
    img_eq = cv2.equalizeHist(img)

    # Sauvegarder image
    name = path.split("/")[-1]
    cv2.imwrite("egalise_" + name, img_eq)

    # Histogramme après égalisation
    plt.hist(img_eq.ravel(), 256)
    plt.title("Histogramme egalise")
    plt.show()

    # Min et Max
    Imin = img_eq.min()
    Imax = img_eq.max()

    print("Imin =", Imin)
```

```
    print("Imax =", Imax)

    # Contraste Michelson
    michelson = (Imax - Imin) / (Imax + Imin)
    print("Michelson =", michelson)

    # Contraste RMS
    mean = np.mean(img_eq)
    rms = np.sqrt(np.mean((img_eq - mean)**2))
    print("RMS =", rms)
```

### 4. Résultats

Les résultats montrent que :

- l'histogramme original est souvent concentré dans certaines valeurs de gris
- après l'égalisation, l'histogramme est mieux réparti
- l'image devient plus claire et les détails sont mieux visibles

Les nouvelles images sont sauvegardées après traitemen

### 5. Commentaires et conclusion

L'égalisation d'histogramme permet d'améliorer le contraste global d'une image en redistribuant les niveaux de gris.

Comparée à l'étirement d'histogramme, cette méthode donne souvent de meilleurs résultats lorsque l'image possède une distribution de niveaux de gris irrégulière.

## Exercice 1 Partie 2 :

### 1. Problème

L'objectif de cet exercice est d'étudier l'impact des **dégradations d'images** sur une image en niveaux de gris (`cameraman.tif`). Deux types de bruit sont considérés :

1. **Bruit gaussien** : ajout d'une perturbation distribuée normalement sur tous les pixels.
2. **Bruit poivre-et-sel** : modification aléatoire d'un pourcentage de pixels en noir ou blanc.

### 2. Solution

- Lire l'image avec OpenCV en niveaux de gris.

- Ajouter un bruit gaussien de variance $\sigma^2$.
- Ajouter un bruit poivre-et-sel avec un pourcentage p de pixels.
- Afficher et comparer les images originales et bruitées

### 3. Code

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

I = cv2.imread("D:/TP VO/TP1//Filtres_Spatials/images/cameraman.tif", cv2.IMREAD_GRAY-
SCALE)

# Vérifier l'image
plt.imshow(I, cmap='gray')
plt.title("Image originale")
plt.axis('off')
plt.show()

#Ajouter du bruit gaussien
def add_gaussian_noise(image, sigma2):
    # image normalisée entre 0 et 1
    image_norm = image / 255.0
    gauss = np.random.normal(0, np.sqrt(sigma2), image_norm.shape)
    noisy = image_norm + gauss
    # revenir à l'échelle 0-255
    noisy = np.clip(noisy, 0, 1) * 255
    return noisy.astype(np.uint8)

I2 = add_gaussian_noise(I, sigma2=0.01)

plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(I, cmap='gray')
plt.title("Originale")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(I2, cmap='gray')
plt.title("Bruit Gaussien σ²=0.01")
plt.axis('off')
plt.show()

# ajouter du bruit poivre et sel
```

```python
def add_salt_pepper_noise(image, amount):
    noisy = image.copy()
    # nombre de pixels à modifier
    num_salt = np.ceil(amount * image.size * 0.5).astype(int)
    num_pepper = np.ceil(amount * image.size * 0.5).astype(int)

    # pixels "sel" (blanc)
    coords = [np.random.randint(0, i-1, num_salt) for i in image.shape]
    noisy[coords[0], coords[1]] = 255

    # pixels "poivre" (noir)
    coords = [np.random.randint(0, i-1, num_pepper) for i in image.shape]
    noisy[coords[0], coords[1]] = 0

    return noisy

I3 = add_salt_pepper_noise(I, amount=0.05)

plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(I, cmap='gray')
plt.title("Originale")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(I3, cmap='gray')
plt.title("Bruit Poivre et Sel p=0.05")
plt.axis('off')
plt.show()


#comparison des trois images
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
plt.imshow(I, cmap='gray')
plt.title("Image originale")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(I2, cmap='gray')
plt.title("Bruit Gaussien σ²=0.01")
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(I3, cmap='gray')
plt.title("Bruit Poivre et Sel p=0.05")
plt.axis('off')
```

```
plt.show()
```

4. **Résultats**

- **Bruit gaussien** : image légèrement floue, grain fin.
- **Bruit poivre-et-sel** : pixels noirs et blancs aléatoires, points visibles.

5. **Commentaires et conclusion**

- Le bruit gaussien affecte tous les pixels, le bruit poivre-et-sel seulement quelques-uns.
- Pour réduire le bruit : filtres moyenneur/gaussien pour le bruit gaussien, filtre médian pour poivre-et-sel

**Exercice 1 partie 2.2:**

**1. Problème**

L'objectif de cet exercice est d'étudier l'effet des filtres de lissage spatial sur une image bruitée.

- Image utilisée : **cameraman.tif**
- Bruit ajouté : **gaussien, variance $\sigma^2 = 0{,}01$**
- Filtres appliqués :
  1. Filtre moyenne 5×5
  2. Filtre 5×5 avec facteur de normalisation 1/81

Le but est de **comparer visuellement et numériquement** les résultats et d'analyser les différences entre ces filtres.

**2. Solution**

1. **Lecture de l'image** en niveaux de gris.
2. **Ajout du bruit gaussien** à chaque pixel.
3. **Application des filtres** :
   - Filtre moyenne 5×5
   - Filtre 5×5 avec facteur 1/81
4. **Comparaison des résultats** :
   - Affichage des images filtrées
   - Soustraction des images pour visualiser les différences

Cette méthode permet d'analyser l'efficacité des filtres pour réduire le bruit et observer l'effet sur les détails de l'image.

### 3. Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import convolve2d

# lire image
I = cv2.imread("D:/TP VO/TP1//Filtres_Spatials/images/cameraman.tif", 0) / 255.0

# ajouter bruit gaussien
sigma2 = 0.01
noise = np.random.normal(0, np.sqrt(sigma2), I.shape)
I_noisy = I + noise
I_noisy = np.clip(I_noisy, 0, 1)

# filtre moyenne 5x5
kernel_avg = np.ones((5,5)) / 25
I_avg = convolve2d(I_noisy, kernel_avg, mode='same')

# masque donné (1/81)
kernel_custom = np.ones((5,5)) / 81
I_custom = convolve2d(I_noisy, kernel_custom, mode='same')

# différence entre les deux images
diff = I_avg - I_custom

# affichage
plt.figure(figsize=(10,6))

plt.subplot(2,2,1)
plt.imshow(I, cmap='gray')
plt.title("Image originale")

plt.subplot(2,2,2)
plt.imshow(I_noisy, cmap='gray')
plt.title("Image bruitée")

plt.subplot(2,2,3)
plt.imshow(I_avg, cmap='gray')
plt.title("Filtre moyenne 5x5")

plt.subplot(2,2,4)
plt.imshow(I_custom, cmap='gray')
plt.title("Filtre masque 1/81")

plt.figure()
```

```
plt.imshow(diff, cmap='gray')
plt.title("Soustraction des deux images")

plt.show()
```

**4. Résultats**

- **Image originale** : claire, sans bruit
- **Image bruitée** : présence de bruit gaussien
- **Filtre moyenne 5×5** : bruit réduit efficacement mais léger flou
- **Filtre 1/81** : bruit réduit moins, détails mieux conservés
- **Soustraction** : met en évidence les zones où les filtres diffèrent

**5. Commentaires et conclusion**

- Les filtres de lissage réduisent le bruit en moyennant les pixels voisins.
- **Filtre moyenne 5×5** : suppression efficace du bruit, perte de détails.
- **Filtre 1/81** : moins efficace pour le bruit, mais préserve les détails.
- **Compromis** : réduction du bruit vs conservation des détails.
- Pour des résultats plus précis selon le type de bruit, on peut utiliser :
  - Filtre **gaussien**
  - Filtre **médian**

**Exercice 2 partie 2.2:**

**1. Problème**

L'objectif de ce TP est d'étudier l'effet des filtres de lissage sur une image bruitée et de mesurer leur efficacité avec le PSNR. Nous utilisons l'image **cameraman.tif** et ajoutons différents types de bruit : sel et poivre, gaussien et speckle.

**2. Solution**

L'image est lue en niveaux de gris et bruitée selon le type choisi. Ensuite, différents filtres sont appliqués : moyenne, gaussien et médian, pour plusieurs tailles (3×3, 5×5, 7×7). Pour chaque

filtre, nous calculons le PSNR pour comparer la qualité des images filtrées par rapport à l'originale. On teste aussi l'effet d'appliquer plusieurs fois un même filtre.

## 2. Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Fonction PSNR
def PSNR(original, denoised):
    mse = np.mean((original - denoised) ** 2)
    if mse == 0:
        return 100
    return 10 * np.log10(1 / mse)

# Bruit sel et poivre
def salt_pepper_noise(image, p):
    noisy = image.copy()
    total_pixels = image.size

    num_salt = int(p * total_pixels / 2)
    num_pepper = int(p * total_pixels / 2)

    # sel
    coords = [np.random.randint(0, i-1, num_salt) for i in image.shape]
    noisy[coords[0], coords[1]] = 1

    # poivre
    coords = [np.random.randint(0, i-1, num_pepper) for i in image.shape]
    noisy[coords[0], coords[1]] = 0

    return noisy


img = cv2.imread("D:/TP VO/TP1//Filtres_Spatials/images/cameraman.tif", cv2.IM-
READ_GRAYSCALE)
img = img / 255.0


noisy = salt_pepper_noise(img, 0.05)

sizes = [3,5,7,9]

psnr_avg = []
psnr_gauss = []
psnr_median = []
```

```
for s in sizes:

    # filtre moyenne
    avg = cv2.blur(noisy,(s,s))
    psnr_avg.append(PSNR(img,avg))

    # filtre gaussien
    gauss = cv2.GaussianBlur(noisy,(s,s),0)
    psnr_gauss.append(PSNR(img,gauss))

    # filtre median
    median = cv2.medianBlur((noisy*255).astype(np.uint8),s)
    median = median/255.0
    psnr_median.append(PSNR(img,median))


plt.plot(sizes, psnr_avg, marker='o', label="Filtre moyenne")
plt.plot(sizes, psnr_gauss, marker='o', label="Filtre gaussien")
plt.plot(sizes, psnr_median, marker='o', label="Filtre médian")

plt.xlabel("Taille du filtre")
plt.ylabel("PSNR (dB)")
plt.title("PSNR en fonction de la taille du filtre")

plt.legend()
plt.grid()
plt.show()
```

### 3. Résultats

Pour le bruit sel et poivre, le filtre médian est le plus efficace, tandis que la moyenne et le gaussien réduisent moins le bruit et floutent l'image. Pour le bruit gaussien, les filtres moyenne et gaussien donnent de bons résultats, alors que le médian est moins performant. Pour le bruit speckle, le filtre gaussien est le plus adapté. L'application répétée d'un filtre moyenne réduit le bruit mais augmente le flou.

### 5. Commentaires et conclusion

Le choix du filtre dépend du type de bruit : médian pour sel et poivre, moyenne ou gaussien pour gaussien, et gaussien pour speckle. Il faut toujours trouver un compromis entre la réduction du bruit et la conservation des détails. Le PSNR permet de quantifier la qualité, mais l'observation visuelle reste importante.